

APPLICATION SECURITY BASELINE AUDIT

Azari Residences / Azari Luxury Properties LTD

B+	84 / 100
SECURITY GRADE	WEIGHTED SCORE

Standing: Good application-security baseline; suitable for controlled production deployment after environment verification and priority hardening.

Prepared for	Azari Residences / Azari Luxury Properties LTD
Prepared by	Zivora Systems
Assessment date	29 July 2026 (Africa/Lagos)
Repository	wbizmo/azari-residences (GitHub)
Audited revision	ed804957576f0136f8ec266373fc733d4f855a73
Method	Manual source review, route/control tracing, configuration review, and delivered regression-test evidence

Classification: Confidential - stakeholder and deployment-readiness use

1. Executive summary

Zivora Systems reviewed the delivered Azari Residences application as a conventional Laravel web application for property discovery, bookings, customer accounts, staff operations, payments, support and property-owner workflows. The assessment was intentionally calibrated to the application's ordinary risk profile. It is not treated as a bank, custody platform, critical-infrastructure system or high-assurance identity platform.

The reviewed revision demonstrates multiple independent security layers rather than reliance on a single controller check. Authentication and session middleware, customer/staff separation, granular staff permissions, route-model ownership checks, request validation, login and endpoint throttling, secure session defaults, payment-provider verification, event deduplication, signed attachment URLs, security response headers, non-debug error handling and request correlation are all present in source.

Overall conclusion. The code has a credible and materially useful security baseline. The weighted score is 84/100 (B+). No critical source-level vulnerability was established within the defined scope. The grade is reduced because administrator multi-factor authentication was not evidenced; the Content Security Policy still permits inline scripts/styles; no current software-composition, SAST or DAST output was supplied; and production infrastructure, secrets, provider dashboards and runtime behavior could not be verified from GitHub source.

Decision statement

Decision	Assessment
Production standing	Conditionally acceptable for normal production use, subject to the deployment gates in Section 9.
Immediate blocker	None established from reviewed source. Production secrets, HTTPS, workers, backups and provider verification remain operational gates.
Priority improvement	Add MFA for administrator/staff accounts and record dependency/security scan results for each release.

What the grade means

- B+ / 84: strong baseline controls and generally sound separation of trust boundaries.
- The score is an assessment of the examined revision, not a warranty that the application is vulnerability-free.
- A successful source review does not replace production configuration review, penetration testing, provider-side validation or ongoing monitoring.

2. Scope, methodology and exclusions

Assessment scope

The assessment examined the server-side Laravel application and representative security-critical workflows at the named Git commit. Review emphasis followed OWASP ASVS concepts and the OWASP Top 10:2025 risk categories, with particular weight on broken access control, security misconfiguration, authentication, injection resistance, integrity of payment/state transitions, logging and operational resilience.

Activities performed

- Confirmed repository identity, default branch, release commit and framework/dependency declarations.
- Traced public, customer, staff, administrator, payment, support and property-owner route boundaries.
- Inspected authentication, role/permission, ownership, session, security-header and correlation middleware.
- Reviewed representative payment initiation/callback/webhook behavior, owner withdrawals, protected downloads, validation and failure handling.
- Considered the delivered regression result: 158 tests passed with 855 assertions after the final factory correction and build verification.
- Compared observed controls against current OWASP guidance and applicable payment-security scoping principles.

Explicit exclusions requested by the client

Excluded Item	Treatment in this report
KYC / identity-verification adequacy	Excluded entirely from scoring and findings. Identity documents are not assessed for regulatory sufficiency, authenticity or KYC compliance.
Trusted-proxy configuration	Excluded from scoring and findings. The wildcard proxy setting is treated as an intentional local/framework compatibility choice, not a report finding.
Previously discussed controller/functional design points	Excluded where identified as intentional application behavior and already protected by the application's RBAC/ownership model.

Other boundary limitations

- No live production penetration test, authenticated dynamic scan, browser attack simulation or network scan was performed.
- No production database, server, DNS, TLS, WAF/CDN, object storage, queue worker, cron, email/SMS account or payment-provider dashboard was accessed.
- No secrets were requested or tested. The review cannot confirm that production values match the secure examples.
- Third-party provider security and availability are outside the code-delivery assessment.
- Package versions were read from source, but no independently executed vulnerability database scan was available for this report.

3. Examined evidence and files

The following inventory identifies the actual source artifacts used to support the conclusions. It is representative of the security-critical code path and is not a claim that every line in the repository was manually reviewed.

Area	Evidence examined
Application and configuration	composer.json; phpunit.xml; .env.example; bootstrap/app.php; config/session.php
Route boundaries	routes/web.php; azari-booking-sprints-05-06.php; azari-sprints-07-08.php; azari-sprints-09-12.php; azari-sprints-13-16.php; azari-property-owner-extension.php
Authentication and sessions	Auth/AzariAdminLoginController.php; framework authentication/Breeze route use; session configuration
Authorization	EnsureAzariStaff.php; EnsureAzariCustomer.php; EnsureStaffPermission.php; EnsureRouteModelOwnership.php; Models/User.php
Headers and diagnostics	SecurityHeaders.php; CorrelationId.php; exception rendering in bootstrap/app.php
Payments	PublicSite/PaymentCheckoutController.php; payment route throttles; provider callback/webhook entry points and event handling
Customer/support access	Account route groups; signed support attachment routes; ownership middleware coverage
Property owner workflows	User/PropertyOwnerController.php; Admin/OwnerMarketplaceController.php; owner route permissions and validation
Delivery evidence	Latest GitHub commit metadata; production asset build output; final application test result of 158 tests / 855 assertions

Revision integrity

All GitHub file evidence cited in this report was requested against commit ed804957576f0136f8ec266373fc733d4f855a73, whose commit message is chore(release): prepare production copy and responsive fixes. This prevents later repository changes from silently altering the basis of the grade.

Observed technology baseline

Component	Observed declaration	Security relevance
PHP	^8.3	Modern supported language baseline; runtime patch level remains a deployment check.
Laravel	^13.8	Framework authentication, validation, CSRF, ORM and middleware controls are used.
Laravel Breeze	^2.4	Standard authentication scaffolding reduces custom-auth risk.
PHPUnit	^12.5.12	Feature and architecture suites are configured and executed.
Sessions	Database, encrypted, Secure, HttpOnly, SameSite=Lax	Strong default cookie and server-side session posture when production matches example config.

4. Scoring model and result

Scoring is weighted to the application's real attack surface. Each domain receives a maximum score. Credit is awarded only where evidence was visible in the audited revision or in the delivered verification result. A missing production observation is not automatically a vulnerability, but it limits assurance.

Domain	Weight	Score	Basis
Access control and authorization	20	18	Strong role separation, granular permissions and object ownership checks; minor consistency/coverage assurance remains runtime-dependent.
Authentication and session security	15	12	Throttling, regeneration, invalidation, encrypted secure cookies; no verified staff MFA.
Input validation and injection resistance	15	13	Framework validation, ORM/query builder and constrained enumerations are widely used; no DAST/fuzzing evidence.
Payment and transaction integrity	15	13	Provider re-verification, event deduplication, safe payload storage and reconciliation patterns; provider configuration/runtime not tested.
Security configuration and browser controls	10	8	CSP, HSTS-on-HTTPS, anti-framing, MIME and referrer controls; CSP permits unsafe-inline.
Logging, auditability and error handling	10	9	Request IDs, safe generic errors, login history, audit routes and payment events; alerting/retention not verified.
Dependency and release assurance	10	6	Modern declared versions and 158-test regression evidence; no SCA/SAST/DAST/SBOM result supplied.
Operational resilience controls	5	5	Health, backup/verification routes, queue/cache/database defaults and fail-closed integration flags are present; operational execution remains a deployment obligation.
TOTAL	100	84	Grade B+ - good baseline, conditional production acceptance

Grade bands

Band	Score	Interpretation
A	90-100	Strong assurance with verified release and production controls.
B	80-89	Good baseline; residual hardening and environment verification required.
C	70-79	Usable baseline with material gaps requiring planned remediation.
D	60-69	Weak assurance; production use should be restricted pending remediation.
F	<60	Unacceptable baseline for ordinary production deployment.

Severity definitions

Critical: likely severe compromise or systemic loss. High: substantial unauthorized access, data or transaction impact. Medium: meaningful weakness requiring planned remediation. Low: defense-in-depth or assurance improvement. Observation: deployment or governance condition without a demonstrated code defect.

5. Control assessment

5.1 Access control and trust boundaries

Customer, staff and administrator areas are separated through middleware and route groups. Staff access requires an authenticated, non-suspended staff account. Customer middleware rejects staff crossover. Administrator actions receive additional administrator or named-permission checks. The User model resolves direct permissions, role permissions and module-level manage permissions, with administrators receiving full permission by design.

Object-level authorization is reinforced by `EnsureRouteModelOwnership`, which checks booking references, payments, documents, guests, notifications, sessions, support/service records, listings, agreements, withdrawals and payout profiles against the authenticated user. Representative owner controllers also repeat ownership checks before view or modification. This layering materially reduces insecure direct object reference risk.

Real-world effect: stealing or guessing a database identifier should not be sufficient to access another customer's protected account records, and a staff login does not automatically grant every administrative action.

5.2 Authentication and sessions

The administrator login is throttled at both route and controller levels, supports email or username, regenerates the session after login, rejects non-staff and suspended accounts, records successful staff login history, and invalidates the session plus CSRF token on logout. Password values are cast through Laravel's hashed cast. Production examples select encrypted database sessions and `Secure, HttpOnly, SameSite=Lax` cookies.

Residual: no administrator/staff MFA control was evidenced. For ordinary hospitality operations this is a medium-priority improvement, not evidence of current compromise.

5.3 Validation, output and injection defenses

Representative controllers use Laravel validation with required types, ranges, maximum lengths, accepted values, existence checks and regular expressions. Database access is primarily through Eloquent and the query builder, which parameterize ordinary values. Blade's default escaped output provides a further baseline against reflected/stored script injection where raw rendering is not deliberately used.

No dynamic payload testing was conducted; therefore injection resistance is assessed from coding patterns rather than proven against every endpoint.

5.4 Payment and state integrity

Payment callbacks locate the expected payment/provider record and then ask the provider driver to verify the transaction before applying the final result. Callback events receive deterministic event identifiers and first-or-create deduplication. Only safe payload fields are stored in the representative controller. Webhooks restrict supported providers, apply rate limits, distinguish invalid signatures/malformed payloads and delegate to a processor. The application redirects customers to provider checkout URLs rather than declaring card-field capture in the reviewed controller.

Real-world effect: a browser redirect or callback parameter alone is not treated as proof of payment; server-side provider verification and idempotent event handling reduce forged-success and duplicate-processing risk.

5. Control assessment (continued)

5.5 Browser, transport and response controls

Global middleware sets X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, Cross-Origin-Opener-Policy and a Content Security Policy. HSTS is added for secure requests. The CSP limits default, connection, framing, base and form-action origins to the application, while allowing required image/font sources.

Residual: script-src and style-src allow inline content. This weakens CSP's protection against some injected markup. Migration to nonces/hashes should follow a compatibility inventory rather than a blind removal that breaks production pages.

5.6 Error handling, logging and accountability

When debug mode is disabled, unhandled exceptions are reported internally while users receive a generic message and request identifier. Correlation middleware validates or generates the request ID and returns it in the response. Staff login history, payment events, audit-log routes and backup verification routes provide useful accountability primitives.

Residual: repository code does not demonstrate centralized alert delivery, retention, immutability, on-call response or production log redaction. These remain operational controls.

5.7 Availability and abuse resistance

Login, booking, quotation, payment, callback, webhook, support and review endpoints include targeted rate limits. Queue/database/cache defaults, health checks and backup verification interfaces are present. Payment/provider integration defaults are disabled and HTTPS requirements are declared, reducing accidental live operation with incomplete credentials.

Rate limits must still be validated behind the production edge so the application observes the intended client identity; proxy behavior itself is excluded from this audit by instruction.

5.8 Test and release evidence

The delivered final verification recorded 158 passing tests and 855 assertions. The suite includes feature and architecture tests under PHPUnit with isolated in-memory SQLite execution. This is strong regression evidence for expected application behavior, including access-control architecture, but it is not a substitute for adversarial testing or a production smoke test.

6. Findings and recommendations

No critical or high-severity finding was established within scope. The following items are prioritized improvements and assurance gaps.

ID / severity	Finding and risk	Required action / target
F-01 Medium	Administrator/staff MFA not evidenced A stolen staff password may provide access until account suspension or session response.	Add TOTP or WebAuthn/passkeys for all staff, require recovery-code handling, and enforce step-up authentication for payment, payout, staff and settings actions. Target: Before broad production staff onboarding
F-02 Medium	Release security scanning evidence absent A vulnerable transitive package or unsafe pattern may remain undetected despite passing functional tests.	Add Composer audit/SCA, npm audit or equivalent SCA, secret scanning and a focused SAST job. Retain results per release and fail on agreed severity thresholds. Target: Next release pipeline
F-03 Low	CSP permits unsafe-inline Inline execution allowances reduce the containment value of CSP against some injection paths.	Inventory inline scripts/styles, move them to versioned assets, then adopt nonces or hashes and remove unsafe-inline incrementally. Target: Planned hardening
F-04 Low	No independent dynamic/adversarial test in evidence Source review may miss runtime routing, template, upload, cache, provider and deployment interactions.	Run an authenticated DAST and focused manual test against staging: IDOR, role crossover, CSRF, stored XSS, upload handling, password reset, callbacks and race conditions. Target: Before major public launch or material feature expansion
F-05 Observation	Production configuration and monitoring not verified Secure code can be weakened by incorrect secrets, debug mode, TLS, permissions, workers, backups or provider settings.	Complete and sign the deployment checklist in Section 9; capture screenshots/log evidence without exposing secrets. Target: Deployment gate

Accepted design conditions - not findings

- KYC/identity assurance is excluded and does not affect the score.
- Trusted-proxy configuration is excluded and does not affect the score.
- Intentional public/guest booking functionality is not treated as a vulnerability where the response is deliberately limited and full access is constrained by authenticated ownership or session state.
- Use of 404 responses for some authorization failures is accepted as an anti-enumeration design choice.
- Use of external payment providers is assessed as an architectural control, not as evidence that Azari Residences is PCI DSS compliant.

7. OWASP-oriented risk mapping

This is a pragmatic mapping, not a formal ASVS certification. “Addressed” means meaningful source controls were observed; it does not mean every ASVS requirement was tested.

OWASP Top 10:2025 category	Standing	Observed basis
A01 Broken Access Control	Strongly addressed	Middleware separation, granular permissions, route-model ownership, signed downloads and repeated owner checks.
A02 Security Misconfiguration	Partly/strongly addressed	Debug false in example, generic errors, security headers, integrations disabled by default; production settings unverified and CSP can be improved.
A03 Software Supply Chain Failures	Partly addressed	Modern declared dependencies and lock-based Composer workflow implied; no current SCA/SBOM evidence supplied.
A04 Cryptographic Failures	Addressed at framework level	Hashed passwords, encrypted sessions, HTTPS-only cookie example, HSTS and provider secrets kept in environment declarations; key management not assessed.
A05 Injection	Strongly addressed by pattern	Laravel validation, Eloquent/query builder and escaped templates; no adversarial DAST/fuzzing.
A06 Insecure Design	Addressed	Trust boundaries, payment re-verification, idempotent events, withdrawal review/reconciliation and fail-closed provider enablement.
A07 Authentication Failures	Partly/strongly addressed	Login throttling, session rotation/invalidation, suspension checks and verified-email gates; MFA absent.
A08 Software/Data Integrity Failures	Addressed	Provider verification and event deduplication; release artifact signing/provenance not evidenced.
A09 Logging and Alerting Failures	Partly/strongly addressed	Request correlation, safe errors, login histories, payment events and audit routes; alerting/retention unverified.
A10 Mishandling of Exceptional Conditions	Strongly addressed	Generic production failure responses, reporting, payment callback recovery paths and fail-closed integration behavior.

Compliance interpretation

OWASP ASVS is used here as a verification reference, while OWASP Top 10 is used as a risk-awareness structure. Neither organization certifies this report. This document does not assert ISO 27001, PCI DSS, NDPA, GDPR or any other statutory/regulatory certification. Formal compliance requires organizational policies, contracts, data maps, infrastructure evidence, operational testing and—where applicable—assessment by an authorized or qualified party.

8. Disclosed metrics

Metric	Value	Interpretation
Weighted security score	84 / 100	Manual evidence-weighted baseline score at audited revision.
Grade	B+	Good baseline; production configuration and priority hardening remain.
Critical findings	0	None established within defined source-review scope.
High findings	0	None established within defined source-review scope.
Medium findings	2	MFA and release security scanning assurance.
Low findings	2	CSP hardening and independent dynamic testing.
Observations	1	Production configuration/monitoring verification gate.
Automated regression tests	158 passed	Delivered final run; not independently rerun by the report author against production.
Automated assertions	855	Delivered final PHPUnit result after test-fixture correction.
Audited Git revision	ed804957576f...	Exact evidence anchor on main branch.
Live endpoints tested	0	This was not a live penetration test or DAST engagement.

How the layers operate in production

- Edge/transport: HTTPS plus secure cookies and HSTS protect session transport; browser headers reduce framing, MIME-sniffing and cross-origin exposure.
- Authentication: framework password verification, throttling and session rotation constrain credential attacks and fixation.
- Authorization: route middleware rejects wrong account classes; permissions restrict staff capabilities; ownership checks bind records to customers.
- Transaction integrity: server-side provider verification, event deduplication and controlled reconciliation prevent client-side success claims from becoming authoritative.
- Application safety: validation, parameterized ORM access and escaped templates reduce malformed input, SQL injection and output injection risk.
- Detection and recovery: correlation IDs, login histories, audit/payment events, generic errors, health checks and backup verification support investigation and recovery.

These controls are complementary. A production system remains secure only when the web server, database, queues, filesystem permissions, secrets, payment-provider settings, TLS, backups and monitoring preserve the assumptions made by the application.

9. Production acceptance checklist

The following items should be confirmed by Azari Residences or its hosting operator before or immediately after deployment. Completion evidence should be retained with the release record.

Gate	Required confirmation	Status
Environment	APP_ENV=production; APP_DEBUG=false; strong unique APP_KEY; no real secrets in repository or deployment ZIP.	☐
Transport	Valid HTTPS certificate; HTTP redirected to HTTPS; secure cookies observed; HSTS present on the public domain.	☐
Database	Least-privilege database user; backups encrypted/restricted; migrations completed; restore test recorded.	☐
Filesystem	Public web root points to /public; .env and storage/private are not web-accessible; writable paths are limited to required directories.	☐
Payments	Only intended providers enabled; live keys configured; webhook secrets/signatures enabled; callback/webhook URLs registered; test payment and refund/reconciliation path verified.	☐
Workers/scheduler	Queue worker supervised; scheduled tasks running; failed jobs monitored; booking hold expiry and reminders verified.	☐
Email/SMS	Production credentials configured; no sensitive payload leakage in logs; delivery failures observable.	☐
Accounts	Default/test accounts removed; staff roles reviewed; administrator MFA remediation scheduled; suspended users tested.	☐
Monitoring	Central error/uptime alerts enabled; log retention/access defined; X-Request-ID visible end to end.	☐
Release assurance	158-test suite passes on deployment build; dependency audit and secret scan retained; staging smoke test complete.	☐
Backups	Automated backup schedule enabled; off-host copy available; restoration procedure tested and signed off.	☐
Incident response	Named technical contact, credential-revocation procedure, provider escalation path and breach-response owner documented.	☐

Recommended acceptance language

“The reviewed source revision is accepted as having a good baseline application-security posture for normal hospitality and property-management use, subject to completion of the production acceptance checklist. Residual risks and exclusions are understood, and no claim of penetration-test completion or regulatory certification is made.”

10. Limitations, disclaimer and references

Baseline-only disclaimer

This report is a point-in-time baseline assessment of identified source files and supplied verification evidence. It is not a guarantee, warranty, insurance policy, legal opinion, certification, attestation of compliance or statement that the application is free from vulnerabilities. Security defects may exist in unexamined files, third-party packages, generated assets, infrastructure, integrations, administrator practices or future changes.

Production behavior can reveal risks that are not visible during static review, including reverse-proxy/header behavior, web-server rules, file permissions, race conditions, queue failure, cache/session behavior, provider callback differences, malformed production data, cloud access controls, TLS/DNS configuration, secret exposure, logging leakage and denial-of-service conditions. A material architecture change, new integration, major framework upgrade, security incident or six-to-twelve-month interval should trigger reassessment.

The application owner remains responsible for lawful data processing, regulatory obligations, vendor due diligence, production configuration, security monitoring, backups, access reviews and incident response. Zivora Systems' conclusions apply only to the scope, revision, evidence and exclusions stated in this report.

Reference sources

- OWASP Application Security Verification Standard (ASVS):
<https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Top 10:2025: <https://owasp.org/Top10/2025/>
- OWASP Session Management Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP Authentication Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Secure Coding Practices:
<https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>
- PCI Security Standards Council - Merchant Resources:
<https://www.pcisecuritystandards.org/merchants/>
- PCI Security Standards Council - Standards overview: <https://www.pcisecuritystandards.org/standards/>
- Laravel 13 documentation: <https://laravel.com/docs/13.x>

Source references provide the control framework and industry context. Repository conclusions are based on the commit-specific GitHub evidence listed in Section 3.

Prepared by	Zivora Systems
For	Azari Residences / Azari Luxury Properties LTD
Date	29 July 2026
Assessment ID	AZR-SEC-BASELINE-2026-07-29